

DEPARTMENT OF
INFORMATION TECHNOLOGY AND ELECTRICAL ENGINEERING
Fall Semester 2025

Implementation of a SHA-256 hardware accelerator for the Croc SoC

Bachelor Project



Nikola Tesic
ntesic@student.ethz.ch

26.06.2025

Advisors: Marco Bertuletti, mbertuletti@iis.ee.ethz.ch
Sergio Mazzola, smazzola@iis.ee.ethz.ch
Enrico Zelioli, ezelioli@iis.ee.ethz.ch
Professor: Prof. Luca Benini, lbenini@iis.ee.ethz.ch

Abstract

The increasing volume of sensitive data transmission and the growing demand for heterogeneous System on Chips have necessitated the design of a cryptographic hardware accelerator for the Secure Hash Algorithm 256 (SHA-256) for Croc. To achieve this, I designed the SHA-256 hardware accelerator and subsequently integrated it into the Croc System-on-Chip [1]. This resulted in a functional SHA-256 accelerator, enabling Croc to execute hashing operations in $2.71 \mu\text{s}$ with an increase in Million Gate Equivalents of 27.5%.

Acknowledgments

I would like to express my gratitude to my supervisors for their guidance and encouragement, particularly Marco Bertuletti. Their assistance with any issues I encountered was crucial for the completion of this report.

I would also like to thank my parents and friends for their emotional support and understanding throughout the project.

Declaration of Originality



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

- ☒ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies¹.
- ☐ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Implementation of SHA-256 hardware accelerator for the Gec SoC

Authored by:

If the work was compiled in a group, the names of all authors are required.

Last name(s):

Tesic

First name(s):

Nikola

With my signature I confirm the following:

- I have adhered to the rules set out in the [Citation Guidelines](#).
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date

16.06.2025

Signature(s)

Nikola Tesic

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

¹ For further information please consult the ETH Zurich websites, e.g. <https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html> and <https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html> (subject to change).

Contents

Declaration of Originality	iii
1 Introduction	1
2 Background	3
2.1 Algorithm	3
2.1.1 Contestualization	3
2.1.2 Operating principle of SHA-256	4
2.2 Croc	7
3 SHA256 development for SoC Croc	10
3.1 Integration in the Croc SoC	15
3.1.1 Handshaking	15
4 Results	22
4.1 Comparison	22
4.2 Possible Improvements	24
5 Conclusion and Future Work	25
List of Acronyms	26
List of Figures	28
List of Tables	29
Bibliography	30

Introduction

With the advancement of open-source methodologies in the hardware domain, it has become feasible to design complete microchips without necessarily relying on proprietary tools or Intellectual Properties (IPs)[2]. This development enables a significantly broader community, including researchers and universities, to access these technologies and cultivate their expertise. Consequently, access has become increasingly democratized, expanding opportunities for numerous individuals to engage in microchip design[3].

Furthermore, the capability to develop microchips utilizing open-source tools and IPs facilitates enhanced customization and optimized performance through the freedom to modify every component of the design. This technological evolution has catalyzed the emergence of platforms such as Parallel Ultra Low Power (PULP), which focus on creating comprehensive open-source hardware and software components.

One of their most recent projects is Croc, a System-on-Chip (SoC) developed within the framework of the Very-Large-Scale Integration 2 (VLSI 2) course. The project achieved partial readiness for tape-out during summer 2024 and was subsequently completed by two students[1]. This initiative provides students who have completed the VLSI 2 course with opportunities to contribute additional functionalities or enhance existing chip components.

Among the various technological possibilities, the increasing volume of sensitive data transmission and the growing demand for heterogeneous SoCs have necessitated the design of a cryptographic hardware accelerator for the Secure Hash Algorithm 256 (SHA-256). This implementation incorporates specific limitations that were deliberately established to maintain manageable complexity within the scope of this work. These constraints, together with the operational principles of the SHA-256 algorithm, will be comprehensively examined in Chapter 2. At this stage, it is sufficient to acknowledge that SHA-256 constitutes an irreversible cryptographic function, a fundamental characteristic that underlies its widespread adoption in secure communications.

Through the integration of this new functionality into the SoC, the system demonstrates potential utility in domains such as telecommunications, blockchain technology, and any context where ensuring signal integrity during transmission is paramount.

1 Introduction

The principal contributions of this work comprise:

- **The design of a SHA-256 hardware accelerator:** Development of a comprehensive RTL implementation as an independent cryptographic module, encompassing all requisite components for secure hash computation.
- **The integration into the Croc SoC:** Incorporation of the SHA-256 accelerator into the USER domain of the existing SoC architecture, ensuring appropriate communication protocols with the remainder of the system.
- **The performance evaluation after Place & Route:** Analysis and assessment of the implemented design utilizing open-source synthesis and backend tools, providing comprehensive metrics regarding area, timing, and power consumption.

Background

This chapter presents all the necessary knowledge to understand how the hardware has been designed and integrated. To this end, it is necessary to understand the operation of the SHA-256 algorithm [4] and acquire a general overview of the Croc architecture [1].

2.1 Algorithm

2.1.1 Contestualization

Security Hash Algorithm 256 (SHA-256) is an algorithm that belongs to the SHA-2 family, developed by the National Security Agency (NSA) to ensure the integrity of information transmitted in communications. This type of algorithm has the following properties:

- Irreversible (given the output, it is not possible to retrieve the original input).
- The output has a fixed length, independently from the input.
- A small change in the input produces a completely different output.

In our case, the length of the algorithm's output, called the digest, is fixed at 256 bits. All the algorithms belonging to the SHA-2 family share the same characteristics, differing only in the length of their output.

To better understand how it is used, I will refer to the situation illustrated in Figure 2.1. Let's assume there is a sender and a receiver. They agree on which SHA implementation to use (in this case, SHA-256). The sender transmits a message of arbitrary length along with its digest, which serves as a kind of fingerprint or digital identity of the message.

The receiver, upon receiving both the message and the digest, independently computes the digest of the received message using SHA-256. If the computed digest matches the one received, the receiver can conclude that the message has not been altered during transmission.

2 Background

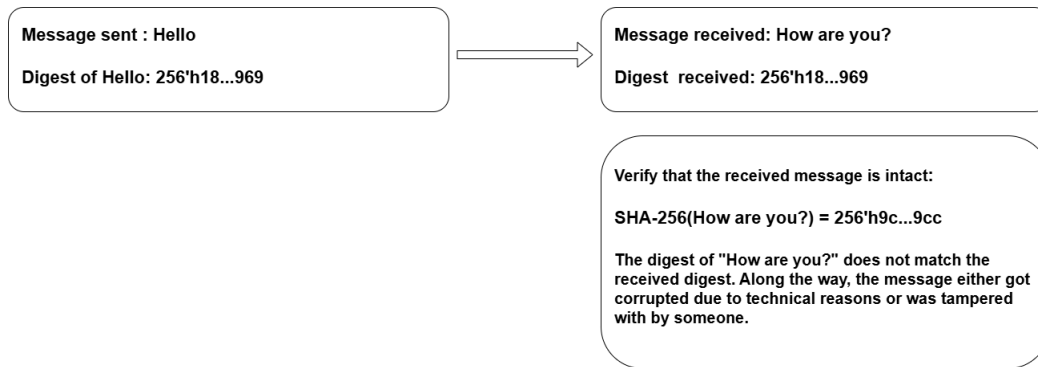


Figure 2.1: The image shows three sections. On the left, it illustrates what the sender transmits; on the right, what the receiver gets and the verification they perform to ensure the received message is correct.

The main purpose of SHA-256 is therefore to ensure the integrity of the message, not its confidentiality.

2.1.2 Operating principle of SHA-256

The operational mechanism of the SHA-256 algorithm can be systematically categorized into the following fundamental processes: input preparation, word expansion, hash computation, chunk transition, and iterative processing.

Input preparation:

The standard SHA-256 algorithm accommodates inputs of arbitrary length through the application of appropriate padding mechanisms to achieve multiples of 512 bits. However, the hardware accelerator presented in this implementation operates exclusively on fixed-length inputs of 512 bits. Without loss of generality, it is assumed that the CPU core manages any requisite padding operations when the input size deviates from the specified length, as this preprocessing phase lacks computational intensity and does not justify hardware acceleration.

The analysis commences with inputs of precisely 512 bits in length. For instance: Input = 512'b000..0010100.

Figure 2.2 illustrates the fundamental algorithmic steps: The 512-bit input (represented by violet bits) undergoes processing to generate a chunk. Chunks maintain a constant length of 512 bits. Since the input bits, combined with the red and green bits displayed at the bottom, cannot be accommodated within a single chunk, an additional chunk must be constructed.

The red bits constitute a fixed separator sequence designed to delimit the input from the green bits. The green bits comprise a 64-bit field representing the input length in big-endian integer format. The maximum representable length approximates 2^{64} bits.

2 Background

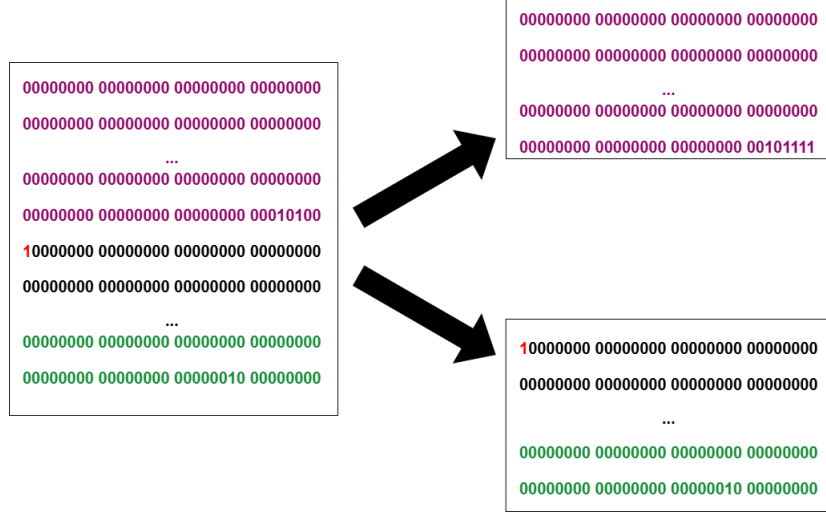


Figure 2.2: The image shows three sections. On the left, it illustrates what the sender transmits; on the right, what the receiver gets and the verification they perform to ensure the received message is correct.

The black bits serve as variable padding elements, consistently set to zero. Given the mandatory 512-bit chunk length requirement, and considering that the red and green bits collectively do not constitute 512 bits, black bits are incorporated to satisfy this constraint. The black bits are positioned exclusively between the red and green bits, as demonstrated in Figure 2.2.

To generalize the procedure: the input is processed, and if the combination of green and red bits can be accommodated within a single chunk, no supplementary chunk is generated. Otherwise, an additional chunk is appended. Scenarios may arise where the input exceeds 512 bits. In such cases, the initial 512 bits are allocated to the preceding chunk, while the remaining bits are positioned in a subsequent chunk alongside the red and green bits.

Expand the words:

The analysis begins with the formal definition of words: these constitute 32-bit segments extracted from the chunk, where the initial 32 bits form the first word. Each chunk comprises precisely 16 words.

Following chunk preparation, the initial chunk undergoes expansion to contain 64 words. The additional words are computed from existing ones according to the following function (with the first word indexed as zero):

$$w[n] = w[n - 16] + \sigma_0(w[n - 15]) + w[n - 7] + \sigma_1(w[n - 2]) \quad \text{where } 15 < n < 64 \quad (2.1)$$

where σ_0 and σ_1 are defined as:

2 Background

$$\sigma_0(w) = (w \ggg 7) \oplus (w \ggg 18) \oplus (w \gg 3) \quad (2.2)$$

$$\sigma_1(w) = (w \ggg 17) \oplus (w \ggg 19) \oplus (w \gg 10) \quad (2.3)$$

A fundamental characteristic of this implementation is that each computed value exhibits dependency on preceding values, an aspect that manifests throughout subsequent algorithmic steps.

Performing the hashing:

Proceeding with the expanded chunk, the subsequent phase involves hash value computation. The hash values represent the initial 32 bits of the fractional components of the square roots of the first eight prime numbers, utilized at algorithm initialization. The implementation follows this structure:

$$h_7 = h_6 \quad (2.4)$$

$$h_6 = h_5 \quad (2.5)$$

$$h_5 = h_4 \quad (2.6)$$

$$h_4 = h_3 + Temp_1(n) \quad (2.7)$$

$$h_3 = h_2 \quad (2.8)$$

$$h_2 = h_1 \quad (2.9)$$

$$h_1 = h_0 \quad (2.10)$$

$$h_0 = Temp_1(n) + Temp_2(n) \quad (2.11)$$

The equations clearly identify the variables that most significantly influence hash value evolution. These two variables are defined as follows:

$$Temp_1(n) = h_7 + \Sigma_1 + Choice + k[n] + w[n] \quad (2.12)$$

$$Temp_2(n) = \Sigma_0 + Majority \quad \text{where } n \in [0 : 63] \quad (2.13)$$

Only $k[n]$ represents a constant variable, determined by prime number derivatives. For example, $k[0]$ corresponds to the fractional component of the cube root of 2 (the first prime number), while $k[n]$ represents the fractional component of the cube root of the $(n+1)$ th prime number. The remaining variables are computed as follows:

$$Choice = (h_4 \wedge h_5) \oplus ((\neg h_4) \wedge h_7) \quad (2.14)$$

$$Majority = (h_0 \wedge h_1) \oplus (h_0 \wedge h_2) \oplus (h_1 \wedge h_2) \quad (2.15)$$

$$\Sigma_1 = (h_4 \ggg 6) \oplus (h_4 \ggg 11) \oplus (h_4 \ggg 25) \quad (2.16)$$

$$\Sigma_0 = (h_0 \ggg 2) \oplus (h_0 \ggg 13) \oplus (h_0 \ggg 22) \quad (2.17)$$

With the exception of $w[n]$ and $k[n]$, all remaining variables exhibit dependency on preceding hash values. This reinforces the previously established principle: the output

2 Background

demonstrates intrinsic dependency on the input. Due to this continuous dependency on antecedent values, even a single-bit modification can result in a fundamentally different computational outcome. The implementation described above is iterated across the entire chunk. Upon completion of the final iteration (constrained by 64 words and 64 constants k), the initial hash value is summed with the final computed value.

Changing the chunk used:

The complete implementation described thus far applies exclusively to the initial chunk. This processing is sufficient when the given input can be accommodated within a single chunk. Otherwise, iterative continuation becomes necessary. The procedure follows a straightforward approach: the subsequent chunk is processed using identical operations as the previous chunk, with one critical distinction. The difference manifests in the hash value computation phase. For the second chunk, the default hash values (the fractional components of the square roots of the first eight prime numbers) are no longer employed. Instead, the final value obtained after 64 iterations on the preceding chunk serves as the new default. This value becomes the new baseline. Therefore, when the final iteration value is added to the initial value, it is summed with the result from the previous chunk. After applying this process to every chunk, the final 256-bit hash value is obtained.

2.2 Croc

Croc is a SoC developed by the PULP team with a purely educational purpose. It is designed to give students attending the VLSI II course at ETH the opportunity to work on a real practical project, thereby developing their skills in hardware design.

The project is the following: students can propose improvements to the existing parts of the SoC or add their own hardware module in the USER domain and complete the backend of the new SoC.

In my case, I chose to develop a cryptographic hardware accelerator based on SHA-256. This approach ensures data integrity verification for all transmitted information while offloading computational burden from the main processor and achieving faster execution times compared to software-only implementations. For this reason, the only part of the SoC that is relevant to us is the one interacting with the USER domain. Figure 2.3 shows a representation of the system.

Core:

The Core, highlighted in red in Figure 2.3, consists of two main components: CVE2 and Core_wrap. CVE2 represents the central processing unit (CPU) of CROC and constitutes the brain of the system. Its task is to manage the entire SoC system and assign various operational tasks. The Core used is specifically the CV32E40P, a 32-bit RISC-V processor that supports IMC extensions (integers, multiplications and compressed instructions), executes programs written in C/assembly, and interacts with memory and peripherals

2 Background

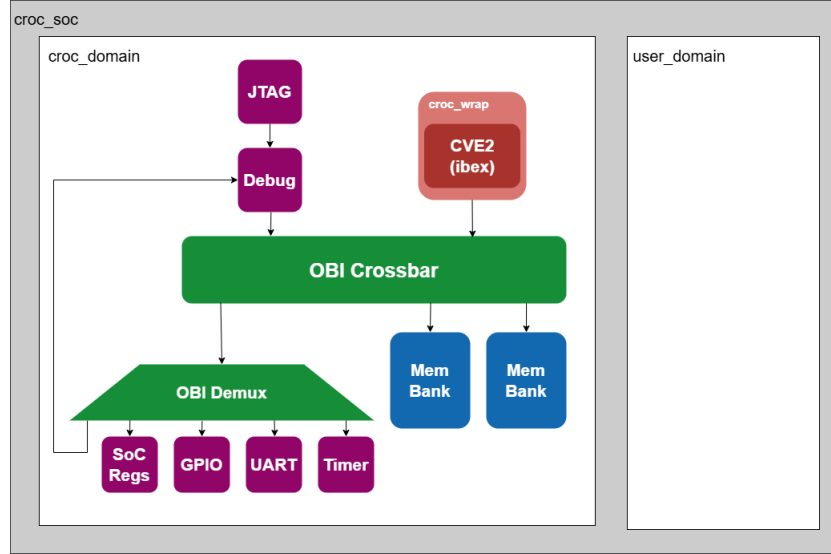


Figure 2.3: Hardware design of Croc

through buses. The task of Core_wrap is instead to act as a "translator" between the CPU and the OBI. To perform this function, it creates two distinct interfaces:

- Instruction Bus (I-side): for instruction fetching.
- Data Bus (D-side): for data load/store operations.

Open Bus Interface (OBI):

The OBI constitutes the interconnection node between the CPU and the system's memories/peripherals. Its main functions include:

- Routing: identifies the origin of requests and determines the appropriate destination
- Arbitration: establishes which master can access a peripheral in case of conflicts
- Handshaking: controls req/gnt and rvalid/rready signals (described in detail in Subsection 3.1.1)

Additionally, the OBI contains all the addresses of the system components, allowing identification of signal destinations. The OBI demux exclusively handles signal routing, while the OBI Crossbar manages the signals themselves, determining access priorities and providing the necessary signals to the demux for routing.

Memory:

In the CROC project, SRAMs (Static Random Access Memory) constitute the main memory of the system and are used to contain:

2 Background

- instructions executed by the CPU (cve2_core)
- temporary data read and written during execution

They therefore represent the equivalent of the "RAM" of a normal microprocessor and are directly accessible by the CPU through the OBI interface. The implemented SRAMs each contain 4 kilobytes (KB), for a total available memory of 8KB. Table 2.1 describes the system memory subdivision.

Address Range	Region	Size	Function
0x0000_0000 - 0x0FFF_FFFF	Peripherals	256 MB	UART, GPIO, Timer, Debug, SoC Ctrl, etc.
0x1000_0000 - 0x1000_1FFF	SRAM	4 KB x 2 banks	Data + instruction memory (SRAM Bank 0/1)
0x2000_0000 - 0x7FFF_FFFF	User space	1.5 GB	Space for user modules or custom peripherals

Table 2.1: Main memory map

Peripherals:

The peripherals implemented in the CROC SoC are six and include the following modules:

Universal Asynchronous Receiver / Transmitter (UART) : Used to transmit and receive serial data (TX/RX) with the outside (therefore used to communicate with other Chips), print messages generated by printf, and communicate with terminal via USB for debugging.

General Purpose Input Output (GPIO): Used to control digital signals at bit level (LEDs, buttons, pins), can be programmed as input or output to interface with external devices.

Timer: Used to count cycles or time, generate periodic interrupts, measure system performance, and trigger timed events.

Debug Module: Used to implement the RISC-V Debug Spec, allow connection via JTAG, control core stop/resume, and access registers and memory remotely.

Joint Test Action Group (JTAG): JTAG is the interface that manages the Debug Module. Therefore, it determines when it is necessary and in which mode to restart or suspend the Core. Furthermore, it can monitor every single operation of the Core and modify its instructions. Essentially, in the case where the Core stops functioning correctly, through JTAG it is possible to restore its proper operation.

Register Interface: Used to connect configurable internal registers to the OBI, map configurations or runtime parameters in a standard way, and integrate with UART, GPIO, or custom modules.

Chapter 3

SHA256 development for SoC Croc

Adding the SHA-256 cryptographic hardware accelerator in the USER domain of Croc involves two macro-phases. The first consists of writing the RTL implementation of the SHA-256 code. After verifying its functionality, it is necessary to add all the structure needed to respect Croc's handshaking.

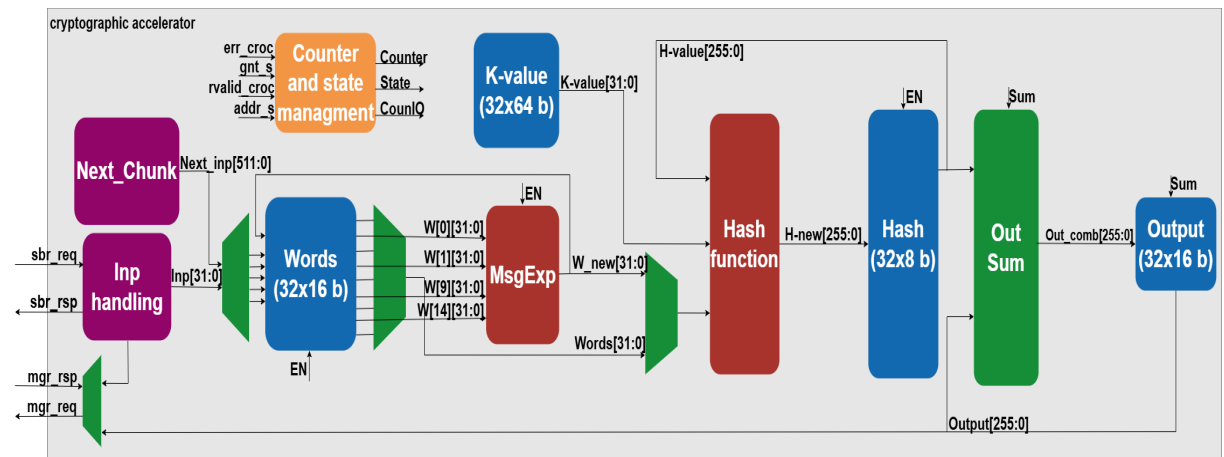


Figure 3.1: Hardware design of cryptographic SHA-256 HW accelerator

During the first phase, signals were received and sent according to other criteria by my testbench. Its operation compared to Croc is simpler. The input was a 512-bit channel and the output was 256-bit.

I would group the entire implemented design into six steps: saving the input, hashing, iteration repetition, sum, chunk change and finally sending the output to the testbench.

Save input:

The first objective consists of dividing the 512-bit input into 16 words and storing them all in memory. As shown in the Algorithm Figure 3.1, a demultiplexer is utilized. The demultiplexer receives two input signals because, after completing the hash implementation on one chunk, it is necessary to execute it on the other as well. Based on the current state, the system decides which chunk to use. If it is in the Idle state, the chunk containing only the input is selected, which has a fixed length of 512 bits. If it is in the Next_chunk state (whose transition will be explained in the subsequent steps), the bits contained in the other chunk are used. This situation is exactly represented in Figure 2.2.

Hashing:

Before illustrating the signal flow, it is necessary to introduce a fundamental element that does not appear in Figure 3.1: the Carry Save Adder (CSA). Pseudocode 1 describes its operation. This module returns two outputs: the partial sum and the carry, which when added together produce the final sum of the three inputs.

Algorithm 1: CSA Module (Carry Save Adder)

```

1: procedure CSA( $XxDI[31:0]$ ,  $YxDI[31:0]$ ,  $ZxDI[31:0]$ )
2:   // Variable declarations
3:    $CFULLxDO \in 2^{32}$ 
4:   // Sum calculation
5:    $SxDO \leftarrow XxDI \oplus YxDI \oplus ZxDI$ 
6:   // Carry calculation
7:    $CFULLxDO \leftarrow (XxDI \wedge YxDI) \vee (XxDI \wedge ZxDI) \vee (YxDI \wedge ZxDI)$ 
8:   // Carry output with left shift
9:    $CxDO \leftarrow \{CFULLxDO[30:0], 0\}$ 
10:  return ( $CxDO, SxDO$ )
11: end procedure

```

Returning to the SHA-256 implementation, after storing the words in memory, it is necessary to implement the SHA-256 function on them. The first step consists of extending the chunk from 16 words to 64 words.

Generally, it is preferable to use a smaller area to reduce costs and implement more functionalities with the same surface. The major problem in terms of area is represented by memory. Consequently, to avoid increasing the required memory, a shift register has been utilized. Its particularity consists in the fact that every time the enable signal (EN) is active, the values in memory are shifted downward: $w[n] = w[n + 1]$, while $w[15] = w_{new}$ (receives the newly calculated value).

The question that might arise is how such a hardware design can function correctly. In subsection 2.1, it was observed that it was first necessary to expand the chunk and subsequently work on the hash values. The reason is as follows: in the first iteration, exactly only the first word is required, in the next iteration the second, and so on. The

signal flow is therefore continuous: it is possible to calculate the new word and use it immediately to calculate the new hashes, save them, and reuse them in the following iteration. It is sufficient to coordinate everything through the signal calculated by the FSM with the EN signal.

A crucial point in this flow is represented by the multiplexer placed between the MsgExp and Hash function modules, connected to another preceding multiplexer. As known, it is necessary to calculate the new words only from the sixteenth onward. Therefore, for the hash, in the first 16 iterations the values are taken directly from the Words memory without passing through the MsgExp module. For this reason, the two multiplexers are necessary.

The other question that might arise is why a shift register is necessary, given that the w signal is used immediately. The answer is found in equation 2.1. As can be observed, with increasing n (moving to the next iteration), the required words are the subsequent ones. For example, if $w[0]$ is needed for $w[16]$, in the next iteration $w[1]$ will be required for $w[17]$. In the proposed hardware implementation, this is handled through constant connections between the module that generates the new word and the registers that, at each cycle, are updated with the content of the following register, as illustrated in Figure 3.1.

The operation of the MsgExp module in Figure 3.1 is described by Pseudocode 2, while the Hash Function module is described by Pseudocode 3..

Pseudocode 2: Message Expansion Module

```

1: procedure MESSAGEEXPANSION( $W[0][31:0]$ ,  $W[1][31:0]$ ,  $W[9][31:0]$ ,  $W[14][31:0]$ )
2:   // Variable declarations
3:    $CSAC1, CSAS1, CSAC2, CSAS2 \in 2^{32}$ 
4:    $D0, D1 \in 2^{32}$ 
5:   // Delta calculations
6:    $D0 \leftarrow (W[0] \gg 7) \oplus (W[0] \gg 18) \oplus (W[0] \gg 3)$ 
7:    $D1 \leftarrow (W[1] \gg 17) \oplus (W[1] \gg 19) \oplus (W[1] \gg 10)$ 
8:   // CSA operations
9:    $(CSAC1, CSAS1) \leftarrow CSA(W[9], W[14], D0)$ 
10:   $(CSAC2, CSAS2) \leftarrow CSA(CSAC1, CSAS1, D1)$ 
11:  // Final addition
12:   $W_{new} \leftarrow CSAC2 + CSAS2$ 
13:  return  $W_{new}$ 
14: end procedure

```

Iteration repetition:

In this section, the focus is on the control signals, composed of few bits but fundamental for the operation of the described implementation. The EN signal coordinates all the previously illustrated elements and is responsible for controlling the flow. The EN signal

Pseudocode 3: Hash Function Module

```

1: procedure HASHFUNCTION( $Words[31:0]$ ,  $K\_value[31:0]$ ,  $H\_values[31:0]$ )
2:   // Variable declarations
3:    $S0xD, MajxD, CHxD, S1xD \in 2^{32}$ 
4:    $CSAC1, CSAS1, CSAC2, CSAS2, CSAC3, CSAS3 \in 2^{32}$ 
5:    $CSAC4, CSAS4, CSAC5, CSAS5, CSAC6, CSAS6 \in 2^{32}$ 
6:   // Hash operations
7:    $S0 \leftarrow (a \ggg 2) \oplus (a \ggg 13) \oplus (a \ggg 22)$ 
8:    $Maj \leftarrow (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c)$ 
9:    $Ch \leftarrow (e \wedge f) \oplus ((\neg e) \wedge g)$ 
10:   $S1 \leftarrow (e \ggg 6) \oplus (e \ggg 11) \oplus (e \ggg 25)$ 
11:  // CSA operations
12:   $(CSAC1, CSAS1) \leftarrow CSA(Words, K\_values, H\_values[7])$ 
13:   $(CSAC2, CSAS2) \leftarrow CSA(CSAC1, CSAS1, S1)$ 
14:   $(CSAC3, CSAS3) \leftarrow CSA(CSAC1, CSAS1, CH)$ 
15:   $(CSAC4, CSAS4) \leftarrow CSA(CSAC2, CSAS2, H\_values[3])$ 
16:  // First assignment
17:   $H\_values[4] \leftarrow CSAC4xD + CSAS4xD$ 
18:  // Additional CSA operations
19:   $(CSAC5, CSAS5) \leftarrow CSA(CSAC3, CSAS3, S0)$ 
20:   $(CSAC6, CSAS6) \leftarrow CSA(CSAC4, CSAS4, Maj)$ 
21:  // Second assignment
22:   $H\_values[0] \leftarrow CSAC6xD + CSAS6xD$ 
23:  // Hash values rotation
24:   $H\_values[7] \leftarrow H\_values[6]$ 
25:   $H\_values[6] \leftarrow H\_values[5]$ 
26:   $H\_values[5] \leftarrow H\_values[4]$ 
27:   $H\_values[3] \leftarrow H\_values[2]$ 
28:   $H\_values[2] \leftarrow H\_values[1]$ 
29:   $H\_values[1] \leftarrow H\_values[0]$ 
30:  return  $H\_new$ 
31: end procedure

```

is determined through the Finite State Machine. In the proposed implementation, two counters are present: one for input and output (described in detail in the following subsection) and another for the hash. The operation is as follows: as soon as the state becomes Hashing, the counter begins to increment. At the 64th iteration, EN returns to zero while the Sum signal is activated. From the name, the signal's intent is intuitive: it activates, as illustrated in Figure 3.1, the Out Sum and Output memory components. This activation lasts exactly one cycle. At the same moment when the sum is performed, the state change from Hashing to Next_Chunk occurs. Since the state change requires one cycle (the signal must first be saved in memory before it can be transmitted), the counter advances and goes from 64 to 65. In the Next_Chunk state, it remains exactly one cycle before returning to the Hashing state (more details on what happens will be provided subsequently). Upon return, the counter is reset to zero and the entire iteration just described is repeated.

To keep track of whether we are working on the first chunk or the second chunk, I have added a one-bit memory that stores a signal called `second_iteration`. When it is zero, it means we are working with the first chunk; when it is one, we are working with the second chunk.

Chunk change:

When the state equals Next_Chunk, the following implementation is actuated: the chunk on which all operations must be performed must be replaced. Therefore, the signal that controls the multiplexer placed in front of the Words memory allows another signal to write to memory. This new chunk is generated by a hardware component that always produces the same value, given that the input length is fixed.

The other relevant event is as follows: the Output memory updates the Hash memory. To accomplish this, the demultiplexer placed in front of the input of the previous hash-values is activated, as illustrated in Figure 3.1.

Sum:

The sum occurs, as previously indicated, at specific moments. This section intends to explain the operation of the involved components. The most relevant component is the demultiplexer with three inputs and one output. Depending on the situation, a different signal writes to memory. In the case of the Idle state it is the hash memory, when the counter has value 64 it is the result of the sum, or it is the memory itself writing its own signal.

An interesting aspect to analyze is the reason why the memory must write to itself and the utility of this implementation. This solution guarantees stable synthesis by always providing a defined behavior at every cycle, avoiding undefined states in simulation and maintaining consistent RAM initialization. This requirement is necessary for every memory and component, since the behavior must always be defined.

Another relevant aspect is understanding why it is necessary to wait for the counter to reach the value 64 and not 63. The reason is due to the memory operation: when the

counter has value 63, all necessary iterations have been completed, however the value has not yet been stored. It must first be saved in the hash memory and subsequently can be used. Therefore, we wait for the counter to reach the value 64, ensuring that the hash-values have been saved in the hash memory.

Sending the output:

The final step consists of sending what has been calculated to the testbench (TB). When the counter reaches the value of 64 and `second_iteration` is one, in addition to performing the sum, the state becomes Output. In this specific case, it is sufficient to compact the data into a single 256-bit signal and send it directly to the TB. In Figure 3.1, this step might not be clear. The memories are all composed of eight 32-bit registers each (except for the word memory, which consists of 16 registers). For this reason, each signal sent or received by the memories in Figure 3.1 is not a single signal but a series of 32-bit signals. As in the case of the hash memory, when it sends the signal, it appears from the image that it sends all 256 bits together; instead, it consists of multiple signals sent in parallel of 32 bits each.

The TB receives the signal and compares it with the signal calculated by the golden model, ensuring they are equal and providing a confirmation message via `printf`.

Testbench and golden model:

As can be inferred from the name, the TB serves to verify whether the implemented design is correct. Its purpose consists of sending the input to the implementation, waiting to receive the done signal (a signal that became active only when the output had to be sent, currently no longer used), and subsequently comparing it with the value calculated by the golden model.

The golden model, receives inputs, processes the output and saves them in a .txt file [5]. From this file, the TB takes the input to send to the implementation and the output with which to compare the result.

In Figure 3.2, a more detailed representation of the operation just described can be found. For further insights on the topic, reference is made to the provided references [6].

3.1 Integration in the Croc SoC

3.1.1 Handshaking

Handshaking is a negotiation process that establishes a connection between two chips. It is a kind of digital "handshake" that ensures both parties are ready and agree on the communication parameters.

Within a SoC, handshaking occurs through two types of signals (a representation of these can be found in Table 3.1 and Table 3.2): one used by the hardware when acting as a manager, and the other when acting as a subordinate. Aside from a few minor

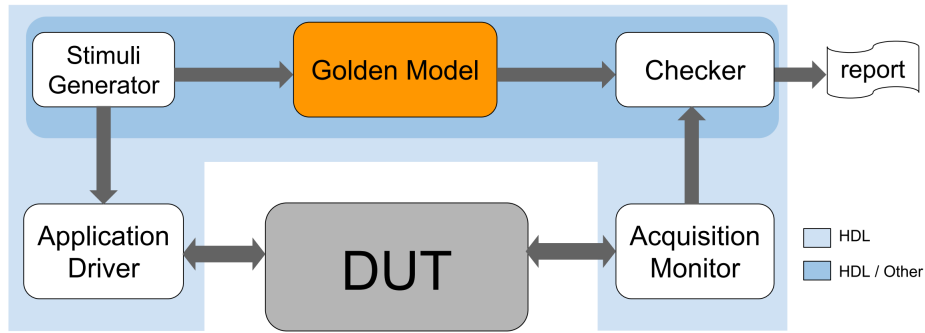


Figure 3.2: Generic Testbench Architecture

differences, the behavior remains essentially the same, whether the core is the manager or my hardware is.

Registers:

The chosen communication type occurs through registers in the main memory. The functionality is as follows: as explained in Table 2.1, inputs and outputs are saved or loaded from SRAM. For communication, registers dedicated to the User space are used, employed to communicate between the two hardware components. In this specific case, four registers are used with an address range from 0x2000_0000 to 0x2000_000C (each register contains 32 bits). The first address is used by the core to indicate where the input begins in SRAM, while the second specifies where the output should be positioned. In both cases, it is assumed that the registers necessary for input and output are sequential (if the first input register is located at 0x1000_0000, the second will be at 0x1000_0004 and so on). The third register serves for the Core to communicate to the hardware that it has completed the operation, by saving the value one. The last register serves instead for the hardware to communicate to the Core the completion of the operation (also in this case by saving the value one).

The reset of the first three registers is determined by the hardware, which executes it when ready to receive new signals, while the fourth is handled by the Core.

It is important to consider that Croc is implemented in such a way as to assign each hardware component in the USER domain a range of 4KB. Therefore, even if many fewer registers are used, this implementation still requires 4KB of space. Requesting less space could cause problems for the OBI and confuse the flow management. Another aspect to consider is that each register used by the OBI is 32 bits. The memories instead, initialized by the hardware and used exclusively by it, can have registers with different lengths and, if implemented internally, use memories of sizes greater or smaller than 4KB. After defining the number of memories and their role, it is also necessary to define how to construct the signals used for communication between them.

Signals implementations:

For this entire section, I will refer to Table 3.1 and Table 3.2.

Name	Bits	Description
req	1	Signals the availability of valid address phase signals.
addr	32	Specifies the address for the transaction.
we	1	Write enable signal (high for writes, low for reads).
be	4	Byte enable signal for specifying active bytes.
wdata	32	Write data (valid only for write transactions).
aid	5/1	Identifier for the address phase transaction.
a_optional	1	Optional signal for additional address phase information.

Table 3.1: Manager signals

Name	Bits	Description
gnt	1	Indicates readiness to accept an address transfer.
rvalid	1	Indicates the validity of response phase signals.
rdata	32	Read data (valid only for read transactions).
rid	5/1	Identifier for the response phase transaction.
err	1	Indicates an error during the transaction.
r_optional	1	Optional signal for additional response phase information.

Table 3.2: Subordinate signals

To send the request, the procedure is as follows. When the master is ready to send the signal, it must set req and wait for the subordinate to respond with gnt equal to one and rid equal to aid (every time the subordinate hardware responds, it must respect this equality). The master hardware will have to wait another cycle to then receive the desired response with rvalid. The gnt bit only indicates that it is ready to receive the signal and therefore prepares the master to receive a response, but does not provide the desired values. Subsequently, whether other signals need to be sent or not, it can lower req.

To avoid complications in the implementation and to prevent data loss, the hardware has been required to set req=0 after receiving a signal with rvalid=1. The problem resided in the sequentiality between gnt and rvalid. First it is necessary to wait for gnt, then rvalid to send a signal; after sending one, if you want to send others consecutively, it may happen that the subordinate hardware is no longer ready. However, one cannot

consider that the signal transition has occurred when gnt was 1, since the rvalid signal, which confirms the acceptance of the signal, has a one-cycle delay compared to gnt. Therefore, the next time you want to send a signal, it is necessary to consider that the last transition has not occurred or maintain the req signal for an additional cycle equal to one. However, it is also necessary to consider that when gnt becomes zero, you cannot keep the req signal raised indefinitely since the Core would interpret it as an error. This information must also be considered when sending a request after a period of inactivity. If raising the req signal does not receive gnt raised, in the following cycle it will be necessary to lower it. The req can remain raised only for one cycle in the case where gnt is zero. Therefore, in the previous case where gnt returned to zero, it was possible to complete the transition.

Given all these issues and considering that more logic involves more space occupation, it has been decided that as soon as the transition is completed, req must return to zero and in the following cycle verify if Croc is available.

For the remaining bits, it is sufficient to consult the table and adapt them based on the operation to be performed, whether reading or writing data. The length of aid and, consequently, also of rid, varies because when the Core writes it can generate multiple types of accesses (instruction fetch, data, atomic transactions, debug, etc.), while our hardware has only one type of request. The err bit in the subordinate might not be clear: it is activated when during the request phase the sequentiality between gnt and req is not respected, or if the request is not supported by the module or aid is not correct.

It is important to keep in mind that the a/r_optional signals must necessarily be implemented, but can remain at zero all the time. This is because it might happen that they are interpreted as undefined and therefore confuse the OBI. Consequently, both functionalities have been implemented in the Idle state, since to start the algorithm it is necessary for the Core to write in the three registers and subsequently, when it has finished, go to read the input. Instead, in the Output state only the phase in which the value is written to SRAM and the completion signal is provided to the Core has been implemented. In Pseudocode 4 the methods for adding all the signals involved in the operation of the four registers can be found, while in Pseudocodes 5 and 6 the necessary elements for input and output implementation can be found.

Algorithm 4: Memory Address Management - Combinational Logic

```

input :Clock signals and control inputs
output:Memory address registers and data outputs

// Initialize next state with current state
1 mem_addr_d  $\leftarrow$  mem_addr_q
2 if start_mem_addr then
    // Reset first three memory address registers
3     mem_addr_d [0]  $\leftarrow$  0 mem_addr_d [1]  $\leftarrow$  0 mem_addr_d [2]  $\leftarrow$  0
4     index_croc  $\leftarrow$  0 index_acc  $\leftarrow$  0 rdata_acc  $\leftarrow$  0
5 end
6 else
    // Handle Croc write operations
7     if err_croc == 0  $\wedge$  check == 0  $\wedge$  req_croc == 1  $\wedge$  addr_croc < 0x2000_000C then
8         index_croc  $\leftarrow$  (addr_croc- 0x2000_0000)  $\gg$  2
9         mem_addr_d [index_croc]  $\leftarrow$  wdata_croc
10    end
11    else
12        index_croc  $\leftarrow$  0
13        mem_addr_d  $\leftarrow$  mem_addr_q
14    end
    // Handle accelerator read operations
15    if check == 1 then
16        index_acc  $\leftarrow$  (addr_acc- 0x2000_0000)  $\gg$  2
17        rdata_acc  $\leftarrow$  mem_addr_q [index_acc]
18    end
19    else
20        index_acc  $\leftarrow$  0
21        rdata_acc  $\leftarrow$  0
22    end
23 end
    // Special handling for register 3
24 if addr_acc == 0x2000_000C  $\wedge$  check == 0 then
25     mem_addr_d [3]  $\leftarrow$  wdata_acc_i
26 end
27 else if we_croc == 1  $\wedge$  err_croc == 0  $\wedge$  addr_croc == 0x2000_000C then
28     mem_addr_d [3]  $\leftarrow$  wdata_croc
29 end

```

Algorithm 5: Idle State Processing

input :Clock signals, control inputs, and interface signals**output**:State updates and memory operations

```

// Idle State Operations
1 hout_d ← ato_h_q
2 if rvalid_i == 1 ∧ err_i == 0 ∧ coun_io_q < 16 then
3   | words_d[coun_io_q] ← rdata_i
4 end
5 addr_s ← 0x2000_0000
6 if gnt_i == 1 ∨ read_q == 1 then
7   | hout_o ← 0
8   | if rvalid_i ∧ ¬err_i ∧ rid_i == 0 ∧ req_o_q == 1 then
9     | // Valid response received
9     | coun_io_d ← coun_io_q + 1 req_o_d ← 0
10  | end
11  | else if rvalid_i == 0 ∧ err_i == 0 ∧ rid_i == 0b11111 ∧ gnt_i == 0 ∧ req_o_q ==
11  | 1 then
11  |   | // Transaction timeout or error
12  |   | coun_io_d ← coun_io_q req_o_d ← 0
13  | end
14  | else
14  |   | // Continue request
15  |   | coun_io_d ← coun_io_q req_o_d ← 1
16  | end
17  | addr_o ← rdata_i + coun_io_q × 4
18  | if addr_s == 0x2000_000C then
19  |   | wdata_acc_i ← 0
20  | end
21 end
22 else
23   | coun_io_d ← coun_io_q
24 end
25 if coun_io_q == 17 then
26   | state_d ← Hashing
27 end

```

Algorithm 6: Output State Processing

input :Clock signals, control inputs, and interface signals

output:State updates and memory write operations

```

// Output State Operations
1 we_o ← 1
2 ato_h_d ← ato_h_new_comb
3 hout_o ← hout_q[coun_io_q]
4 addr_s ← 0x2000_0004
5 addr_o ← rdata_i + coun_io_q × 4
6 if rvalid_i ∧ ¬err_i ∧ rid_i == 0 ∧ req_o_q == 1 then
    // Valid response received
7     coun_io_d ← coun_io_q + 1
8     req_o_d ← 0
9 end
10 else if rvalid_i == 0 ∧ err_i == 0 ∧ rid_i == 0b11111 ∧ gnt_i == 0 ∧ req_o_q == 1
    then
    // Transaction timeout or error
11     coun_io_d ← coun_io_q
12     req_o_d ← 0
13 end
14 else
    // Continue request
15     coun_io_d ← coun_io_q
16     req_o_d ← 1
17 end
18 if coun_io_q == 8 then
    // Output complete - reset and return to Idle
19     start_mem_addr ← 1
20     addr_s ← 0x2000_000C
21     coun_io_d ← 0
22     coun_h_d ← 0
23     req_o_d ← 0
24     irq ← 1
    // Signal completion to CPU
25     wdata_acc_i ← 1
26     ato_h_d ← SHA_IV
27     state_d ← Idle
28 end

```

Results

In this chapter, we will analyze the values obtained after Place and Route (P&R) and compare them with those obtained from Croc without the proposed hardware implementation, evaluating the advantages and disadvantages of the implementation.

4.1 Comparison

Figure 4.1 and Figure 4.2 illustrate the difference between the two implementations. It should be noted that Croc with my hardware accelerator has not completed all backend steps. Therefore, the final results may differ significantly.

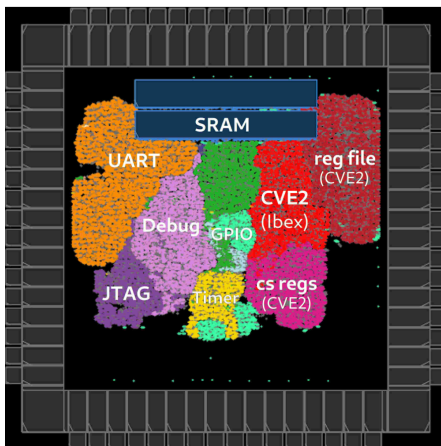


Figure 4.1: Result of base line Croc

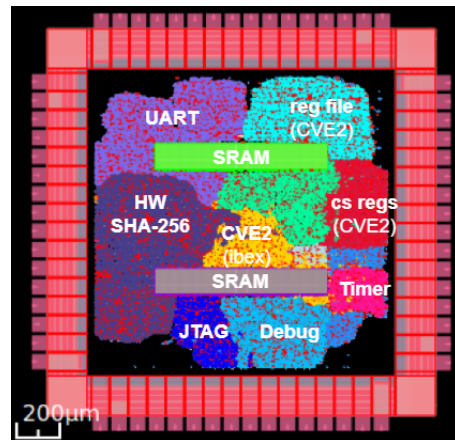


Figure 4.2: Results of Croc with my hardware implementation

4 Results

Metric	Croc without implementation	Croc with implementation	Improvement
Frequency (MHz)	80	80	1.0x
Cycle (cycles)	11683	217	53.8x faster
Time (μ s)	146.0375	2.7125	53.8x faster
Area (kGE)	96.588	124.743	1.27x larger

Table 4.1: Performance comparison between Croc SoC configurations

Before proceeding with the analysis of the results, it is necessary to make three important clarifications.

The first concerns the measurement methodology: to calculate the cycles required to execute SHA-256 on the core, a non-optimized code was used. Therefore, the obtained values represent a conservative scenario that could overestimate the performance of the hardware accelerator. An optimized software implementation of SHA-256, utilizing compiler optimization techniques, algorithmic optimizations specific to the RISC-V architecture, and fully exploiting the processor pipeline, could significantly reduce the number of required cycles.

The second clarification regards the backend design flow. To obtain the presented data, it was necessary to partially complete the backend. During this phase, some signals did not meet timing constraints, making it necessary to evaluate different optimization strategies including: pipelining (if architecturally feasible), synthesis optimizations, floorplanning/placement, physical optimizations, and architectural modifications. To choose the most appropriate optimization, an analysis of critical paths was conducted. After identifying the problematic signals and analyzing their paths, it was chosen to do a floorplanning as you can see in Figure 4.2.

The final clarification concerns the frequency measurement methodology. To measure the obtained value, a parasitic estimate is used, which is derived from average values on the routing layer and the routing distance. No coupling or other complex effects are considered in this approach. This simplified method is used instead of a parasitic extraction engine (PEX), which is significantly more complex and requires a separate calibration file downloaded from the openroad-flow-scripts repository. This methodology was chosen based on the PULP group’s experience with Basilisk [7], where they found that this simplified parasitic estimation approach approximates the final frequency value with approximately 10% error.

Analyzing Table 4.1, the area indicates that the required active area has increased by a factor of 1.27, resulting in an increase in the utilized area. It is important to emphasize that Gate Equivalent (GE) represents the ratio between the active area and the reference unit area (in this case, NAND2x1). This increase does not necessarily imply an increase in the total chip area, as inactive area might be available for use. In this specific case, the total area remained unchanged.

The results demonstrate that the proposed implementation provides an improvement in terms of cycles and execution time, while requiring an increase in active area. Considering that the performance improvement might be overestimated due to the non-

optimized code used as reference, it is believed that this trade-off remains satisfactory even with a more conservative improvement, provided it exceeds 5x in terms of time and execution cycles.

To obtain these results, we utilized the IHP SG13G2 130nm technology with typical-typical (tt) and fast-fast (ff) process corners using OpenROAD version 1.0, where the tt corner represents typical process conditions at 1.20V and 25°C, while the ff corner represents fast process conditions at 1.32V and -40°C. The same conditions used by Croc.

4.2 Possible Improvements

The implementation presents room for improvement, particularly regarding MGE reduction. One possible improvement concerns the reduction of memory usage. The implementation requires approximately 3220 bits in total, of which 2048 are dedicated exclusively to storing K-values. An alternative strategy could consist of implementing the cube root mathematical function in hardware, allowing storage of only the first 64 prime numbers. However, it is not guaranteed that such implementation would result in a reduction of the overall active area.

These represent the main improvements identified for the current implementation. However, the possibility of further optimizations that could emerge from more in-depth analyses or alternative design approaches is not excluded.

Regarding the work in general, a possible improvement would be to complete the backend design flow. The missing part concerns the use of KLayout to verify that the designed hardware meets the foundry requirements, namely compliance with Design Rule Check (DRC) and Layout Versus Schematic (LVS) verification. This final phase is essential to ensure the manufacturability of the design and the correspondence between the physical layout and the logical netlist.

Conclusion and Future Work

The implementation of the SHA-256 hardware accelerator demonstrates a good trade-off between performance and area. The main results are:

- A functional SHA-256 accelerator has been designed in SystemVerilog
- The performance in terms of cycles is 53.8x (value to be considered with caution) better than the pure software implementation
- The design has been synthesized, placed and routed in IHP130nm technology using open source tools, achieving an operating frequency of 80MHz

With only additional 28.155 kGE in the total chip area (27% increase), the design reduces the execution time for SHA-256 by 53.8x (value to be considered with caution). Thanks to this implementation, the Croc SoC will be able to execute SHA-256 operations in significantly reduced time when necessary, while maintaining the ability to handle other operations in parallel.

List of Acronyms

ASICApplication-Specific Integrated Circuit

CPUCentral Processing Unit

CSACarry-Save Adder

CVE2RISC-V Core

DRCDesign Rule Check

ETHEidgenössische Technische Hochschule

FSMFinite State Machine

GEGate Equivalent

GPIOGeneral Purpose Input/Output

HWHardware

ICIntegrated Circuit

IMCInstruction Memory Controller

JTAGJoint Test Action Group

KBKilobyte

LVSLayout Versus Schematic

MEMMemory

List of Acronyms

NLC	Next Linear Congruential
NSA	National Security Agency
OBI	Open Bus Interface
P&R	Place and Route
PEX	Parasitic Extraction engine
PULP	Parallel Ultra-Low-Power
SHA-256	Secure Hash Algorithm 256
SoCs	Systems-on-Chips
UART	Universal Asynchronous Receiver-Transmitter
VLSI	Very Large Scale Integration

List of Figures

2.1	The image shows three sections. On the left, it illustrates what the sender transmits; on the right, what the receiver gets and the verification they perform to ensure the received message is correct.	4
2.2	The image shows three sections. On the left, it illustrates what the sender transmits; on the right, what the receiver gets and the verification they perform to ensure the received message is correct.	5
2.3	Hardware design of Croc	8
3.1	Hardware design of cryptographic SHA-256 HW accelerator	10
3.2	Generic Testbench Architecture	16
4.1	Result of base line Croc	22
4.2	Results of Croc with my hardware implementation	22

List of Tables

2.1	Main memory map	9
3.1	Manager signals	17
3.2	Subordinate signals	17
4.1	Performance comparison between Croc SoC configurations	23

Bibliography

- [1] P. Sauter, T. Benz, P. Scheffler, H. Pochert, L. Wüthrich, M. Povišer, B. Muheim, F. K. Gürkaynak, and L. Benini, “Croc: An end-to-end open-source extensible risc-v mcu platform to democratize silicon,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.05090>
- [2] P. Scheffler, P. Sauter, T. Benz, F. K. Gürkaynak, and L. Benini, “Basilisk: An end-to-end open-source linux-capable risc-v soc in 130nm cmos,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.15107>
- [3] Tiny Tapeout Team, “Tiny tapeout: From idea to chip design in minutes,” <https://tinytapeout.com/>, 2024, accessed: 2025-06-13. [Online]. Available: <https://tinytapeout.com/>
- [4] D. Martin, “Sha-256 algorithm explanation,” 2022, accessed: 2025-06-13. [Online]. Available: <https://github.com/dmarman/sha256algorithm>
- [5] F. K. Gürkaynak, K. Gaj, B. Muheim, E. Homsirikamol, C. Keller, M. Rogawski, H. Kaeslin, and J.-P. Kaps, “Hardware evaluation of SHA-3 3rd round candidates,” <https://iis-people.ee.ethz.ch/~sha3/>, 2016, accessed: 2025-06-14.
- [6] H. Kaeslin, *Top-Down Digital VLSI Design: From Architectures to Gate-Level Circuits and FPGAs*. Burlington, MA: Morgan Kaufmann, 2014, volume 1: Front-End Design.
- [7] P. Sauter, T. Benz, P. Scheffler, M. Povišer, F. K. Gürkaynak, and L. Benini, “Basilisk: A 34 mm² end-to-end open-source 64-bit linux-capable risc-v soc in 130nm bicmos,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.10060>